

Cover Shelter Design

Mehmet Melikşah Çalışkan

Cemal Bolat

November 2025

1 Input Variables

The following sets and parameters are defined as input data to the model:

- V : Set of vertices (container locations).
- T : Set of container types.
- $r \in T$: Residential container type.
- C_t : Maximum number of residential vertices that can be assigned to a building of type $t \in T \setminus \{r\}$.
- D_t : Service distance threshold for type $t \in T \setminus \{r\}$, indicating the maximum distance within which the type can serve residential vertices. Distances follow a mixed Manhattan–Euclidean metric.
- S_t : Number of containers that constitute one complete group of type $t \in T \setminus \{r\}$.
- Δ_t : Maximum allowable distance between any two vertices belonging to the same group of type $t \in T \setminus \{r\}$.
- $d(u, v)$: Mixed Manhattan–Euclidean distance between vertices $u, v \in V$.
- $h(u, v)$: Euclidean distance between vertices $u, v \in V$, used for compactness or clustering penalties.

2 Precomputed Variables

The following auxiliary sets are precomputed based on spatial relationships between vertices and type-specific parameters.

Candidate Subgroups:

$$U(t) = \left\{ U \subseteq V \mid |U| = S_t \right\}, \quad \forall t \in T \setminus \{r\}.$$

Explanation: For each non-residential type t , the set $U(t)$ contains all possible subsets of vertices of size S_t . Each subset represents a potential configuration (subgraph) of containers that could form one complete group of type t .

Feasible Groups Under Diameter Constraint:

$$G_t = \left\{ U \in U(t) \mid d(u, v) \leq \Delta_t, \forall u, v \in U \right\}, \quad \forall t \in T \setminus \{r\}.$$

Explanation: The subset G_t filters $U(t)$ by removing all candidate groups whose internal pairwise distances exceed the maximum allowable limit Δ_t . Thus, G_t includes only compact, spatially feasible groups for each non-residential type.

Neighborhood Groups for Residential Vertices:

$$S_{v,t} = \left\{ G \in G_t \mid \exists u \in G : d(u, v) \leq D_t \right\}, \quad \forall v \in V, \forall t \in T \setminus \{r\}.$$

Explanation: For each vertex v and type t , the set $S_{v,t}$ contains all feasible groups of type t that are within the service distance D_t from v . Intuitively, it defines which type- t facilities (groups) can potentially serve residential container v under the mixed Manhattan–Euclidean distance metric.

3 Decision Variables

All decision variables are binary.

Type Assignment Variables:

$$x_{v,t} = \begin{cases} 1, & \text{if vertex } v \\ & \text{is assigned type } t, \\ 0, & \text{otherwise.} \end{cases} \quad \forall v \in V, \forall t \in T.$$

Explanation: The variable $x_{v,t}$ indicates whether vertex v hosts a container of type t . Each vertex can take at most one type, including the residential type r .

Residential-to-Group Assignment Variables:

$$y_{v,g,t} = \begin{cases} 1, & \text{if residential vertex } v \\ & \text{is assigned to group } g \\ & \text{of type } t, \\ 0, & \text{otherwise.} \end{cases} \quad \forall v \in V, \forall g \in G_t, \forall t \in T \setminus \{r\}.$$

Explanation: The variable $y_{v,g,t}$ links each residential vertex to a feasible group g of a non-residential type t . It determines whether vertex v is served by that particular facility group.

Group Activation Variables:

$$u_{g,t} = \begin{cases} 1, & \text{if group } g \\ & \text{of type } t \text{ is activated,} \\ 0, & \text{otherwise.} \end{cases} \quad \forall g \in G_t, \forall t \in T \setminus \{r\}.$$

Explanation: The binary variable $u_{g,t}$ activates or deactivates group g of type t . Only activated groups can serve residential vertices and contribute to the model's coverage and capacity constraints.

4 Objective

Maximize the total number of residential containers (where r denotes the residential type):

$$\max \sum_{v \in V} x_{v,r}$$

5 Core Constraints

1. Single type per vertex:

$$\sum_{t \in T} x_{v,t} \leq 1 \quad \forall v \in V.$$

Explanation: Each vertex can host at most one container type, including the residential type r .

2. Neighborhood coverage (Rainbow constraint):

$$\sum_{g \in S_{v,t}} u_{g,t} \geq x_{v,r} \quad \forall v \in V, \forall t \in T \setminus \{r\}.$$

Explanation: For every residential vertex v , and for each non-residential type t , at least one activated group $u_{g,t}$ within $S_{v,t}$ must exist within service distance D_t . This ensures that every residential vertex is covered by all required non-residential facilities.

3. Group size consistency:

$$u_{g,t} \leq x_{v,t} \quad \forall t \in T \setminus \{r\}, \forall g \in G_t, \forall v \in g.$$

Explanation: A group g of type t can only be activated if all its member vertices v are assigned as that same type. This enforces internal consistency between group activation and vertex-type assignment.

4. Group–vertex membership coherence:

$$x_{v,t} \leq \sum_{\substack{g \in G_t \\ v \in g}} u_{g,t}, \quad \sum_{\substack{g \in G_t \\ v \in g}} u_{g,t} \leq 1, \quad \forall v \in V, \forall t \in T \setminus \{r\}.$$

Explanation: A vertex v can be assigned to type t only if it belongs to at least one active group of that type, and each vertex can participate in at most one such group. These two inequalities jointly prevent cross-group overlaps and maintain group exclusivity.

6 Tests

Tests are done by four categories: Graph Size, amount of non-residential-types, container sizes of non-residential-types, and whether model prevents empty vertices or not. Black colored vertices are empty vertex when model does not prevent empty vertices

6.1 Graph With Size 6x6 with weight 6

1. One none-residential-type

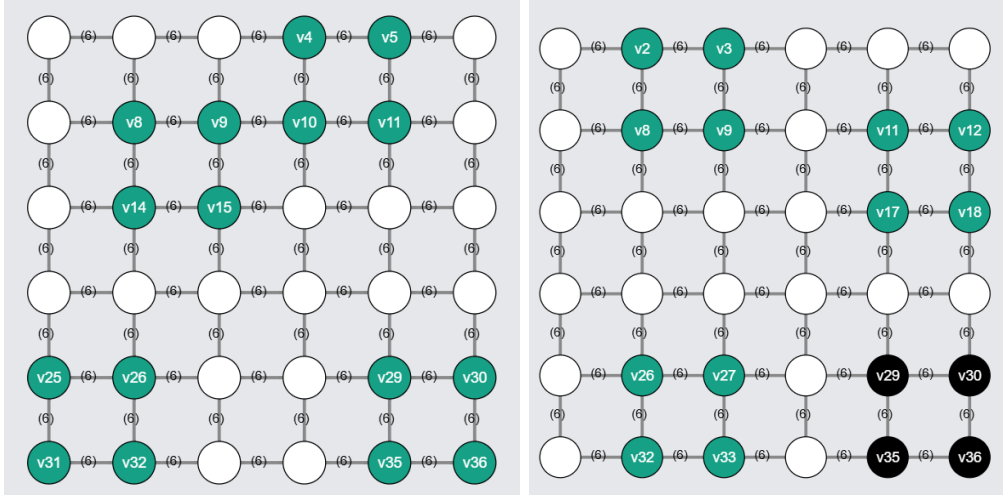


Figure 1: The One none-residential-type with 4 container size

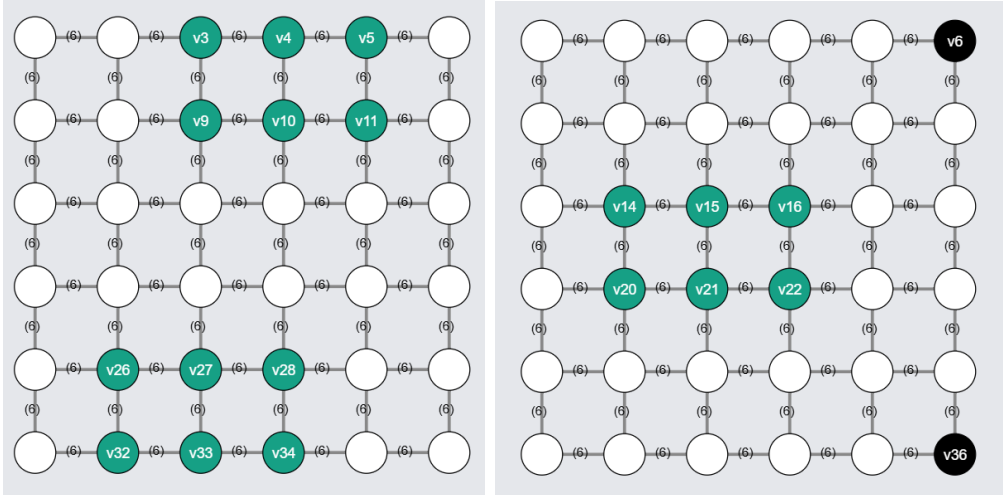


Figure 2: The One none-residential-type with 6 container size

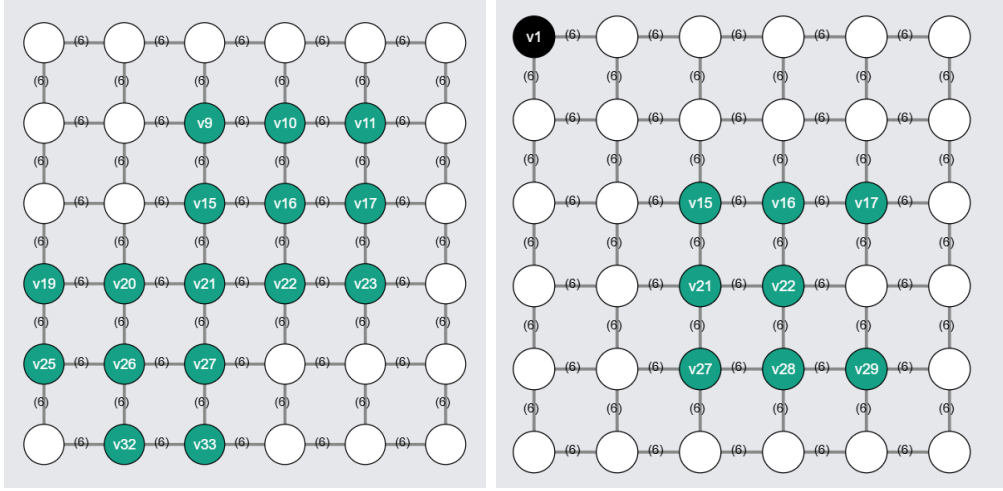


Figure 3: The One non-residential-type with 8 container size

2. Two non-residential-type



Figure 4: The two non-residential-type with 4 and 6 container size



Figure 5: The two none-residential-type with 4 and 8 container size



Figure 6: The two none-residential-type with 6 and 8 container size

3. Three none-residential-type



Figure 7: The two none-residential-type with 4, 6 and 8 container size. Note: when we prevent empty vertices the model could not find any solution for all pair of non-residential-type and residential-type containers, that is why it colored all of them any size suitable non-residential-type container

6.2 Graph With Size 10x10 with weight 10

1. One none-residential-type

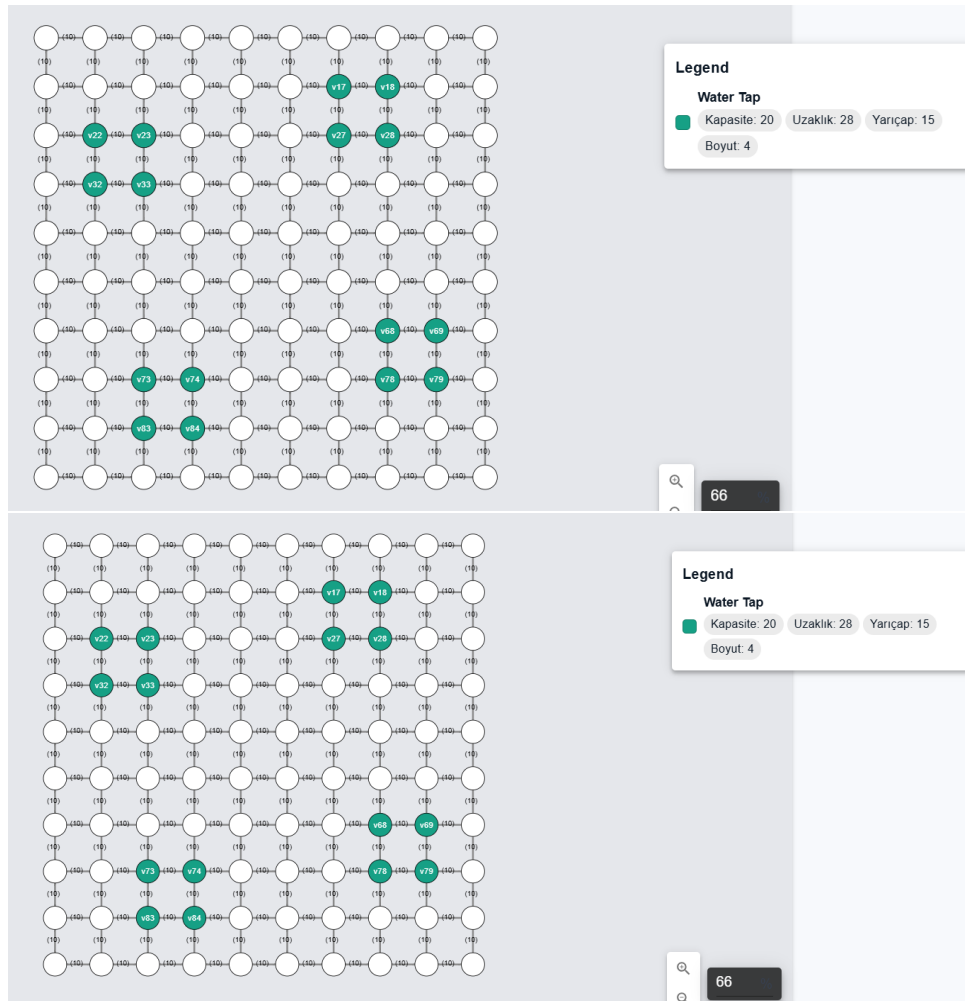


Figure 8: The One none-residential-type with 4 container size

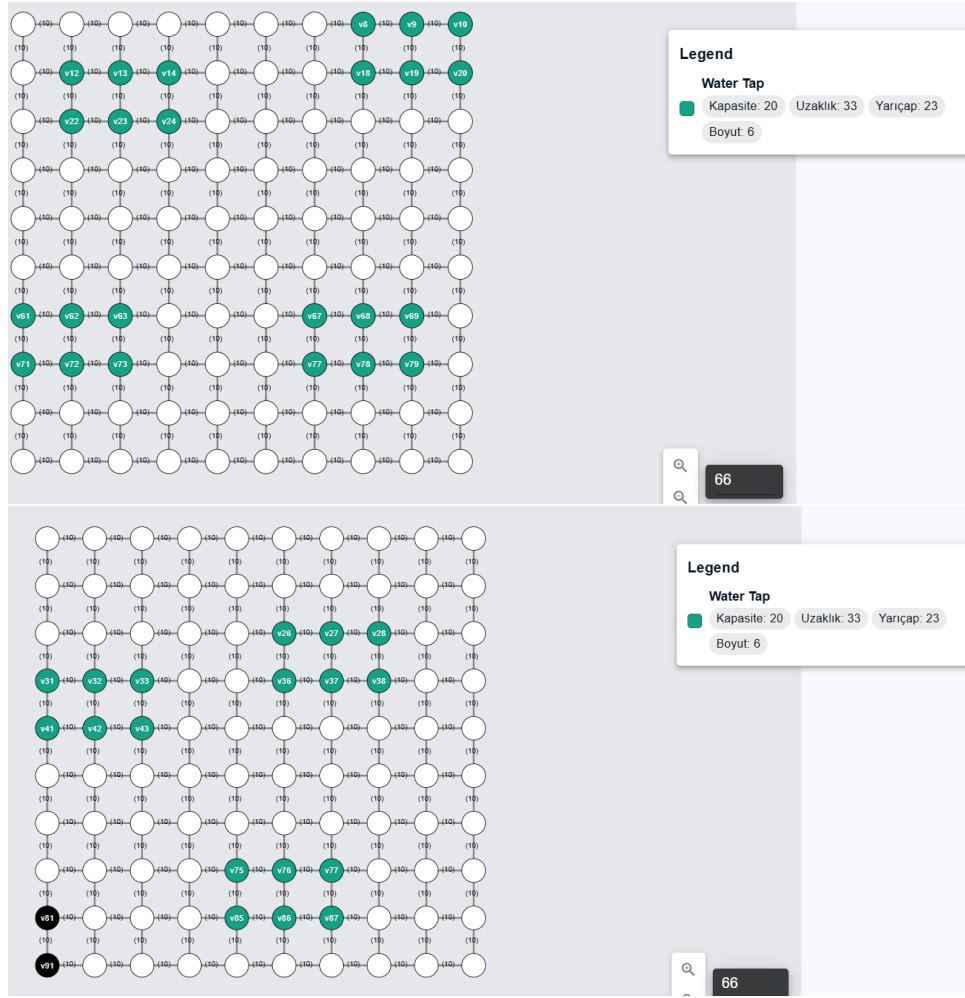


Figure 9: The One none-residential-type with 6 container size

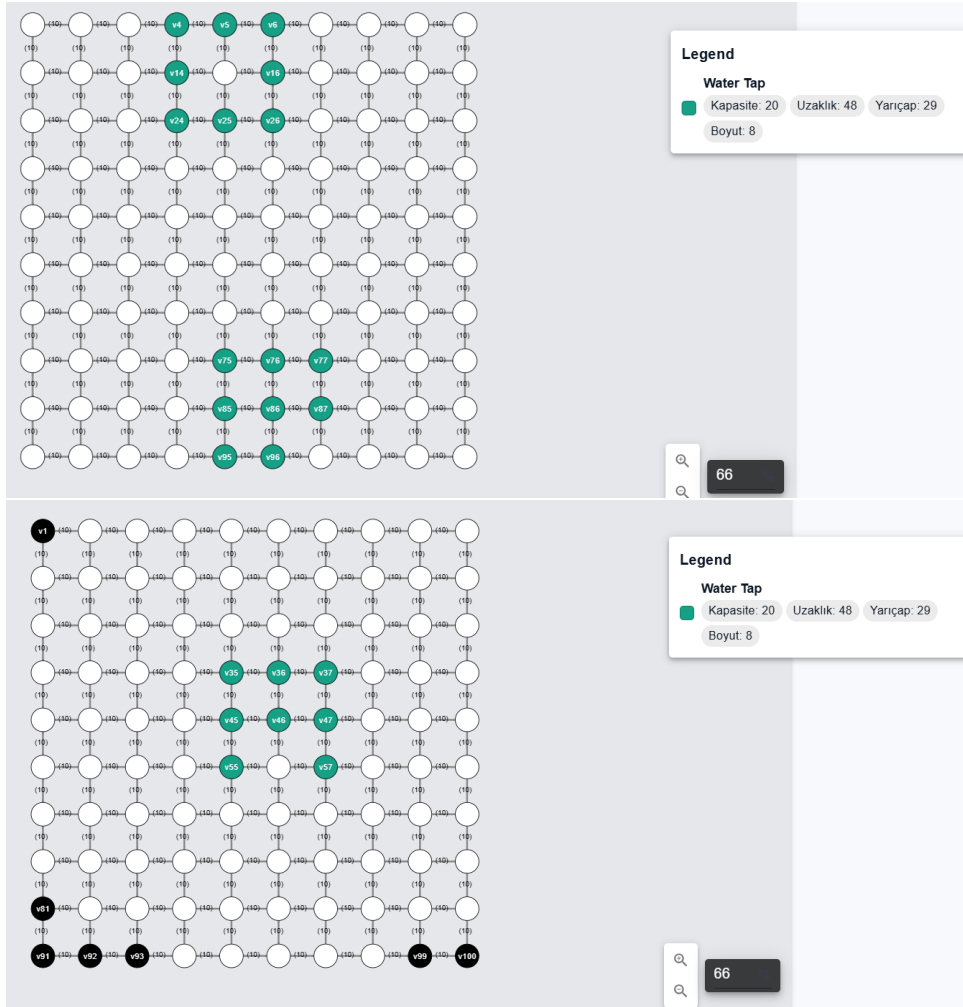


Figure 10: The One none-residential-type with 8 container size

2. Two none-residential-type

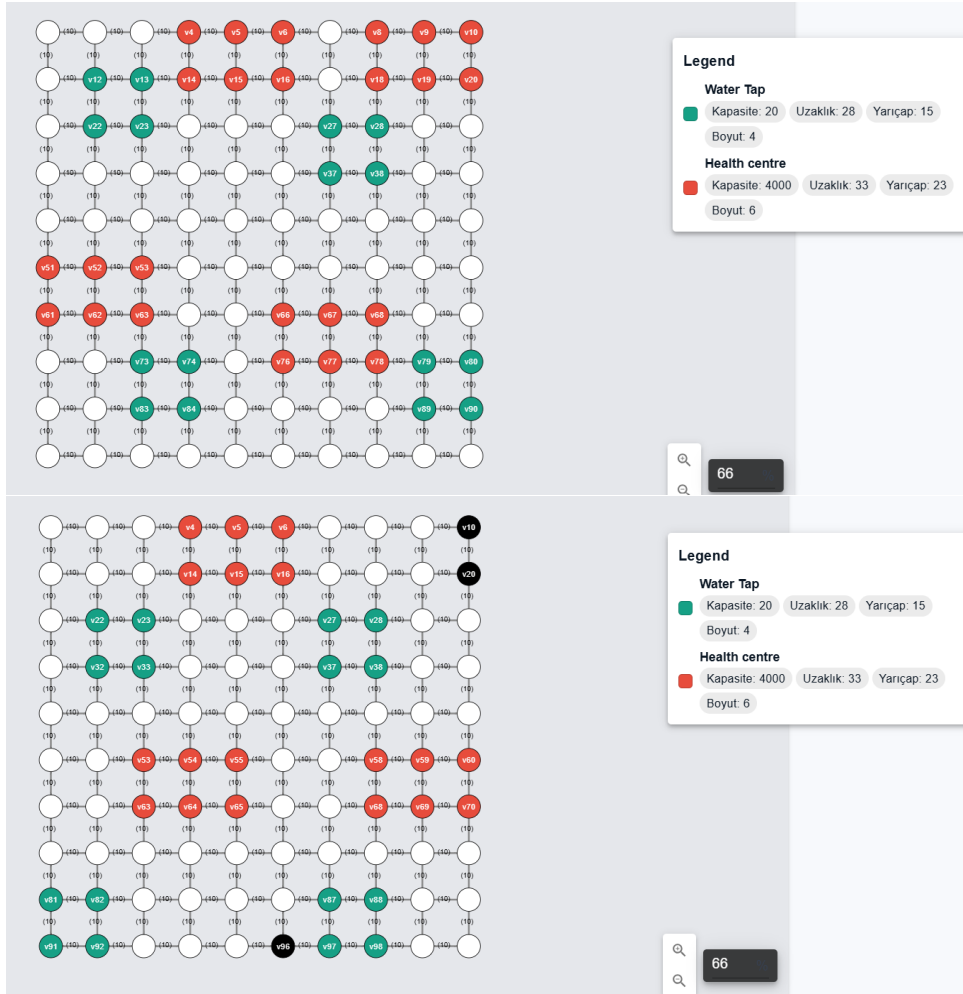


Figure 11: The two none-residential-type with 4 and 6 container size

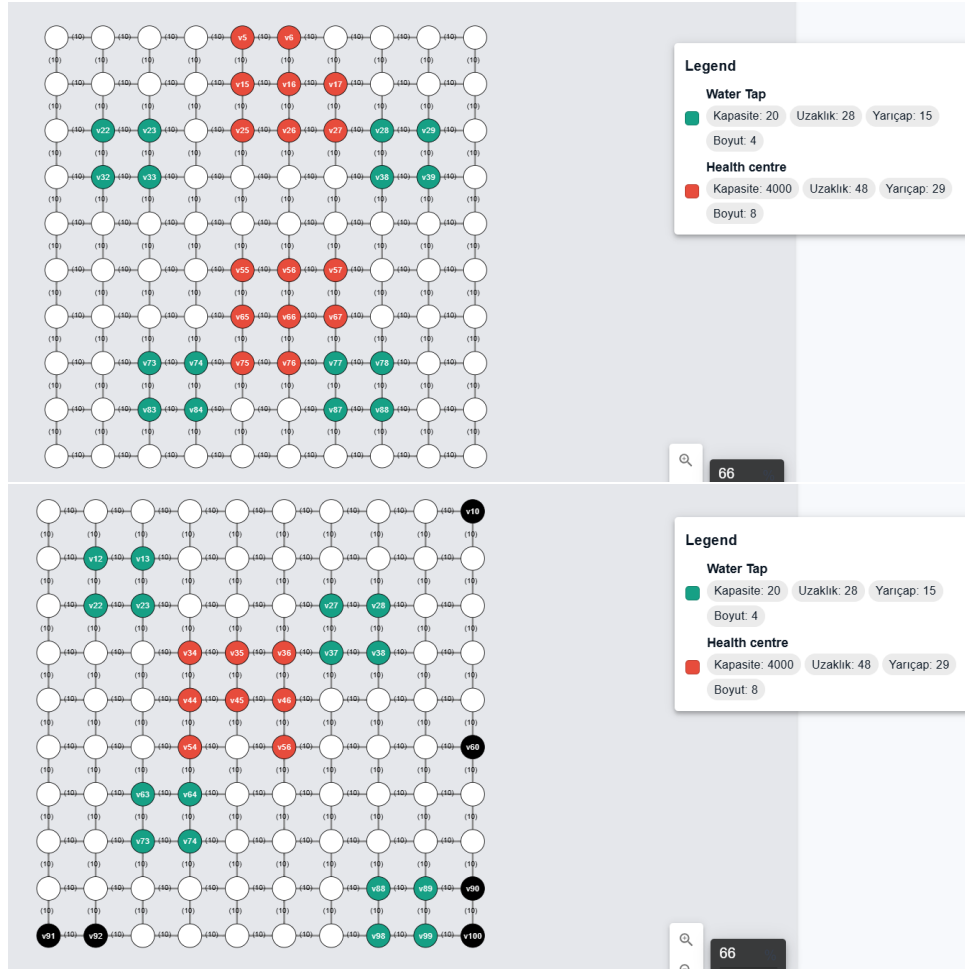


Figure 12: The two non-residential-type with 4 and 8 container size

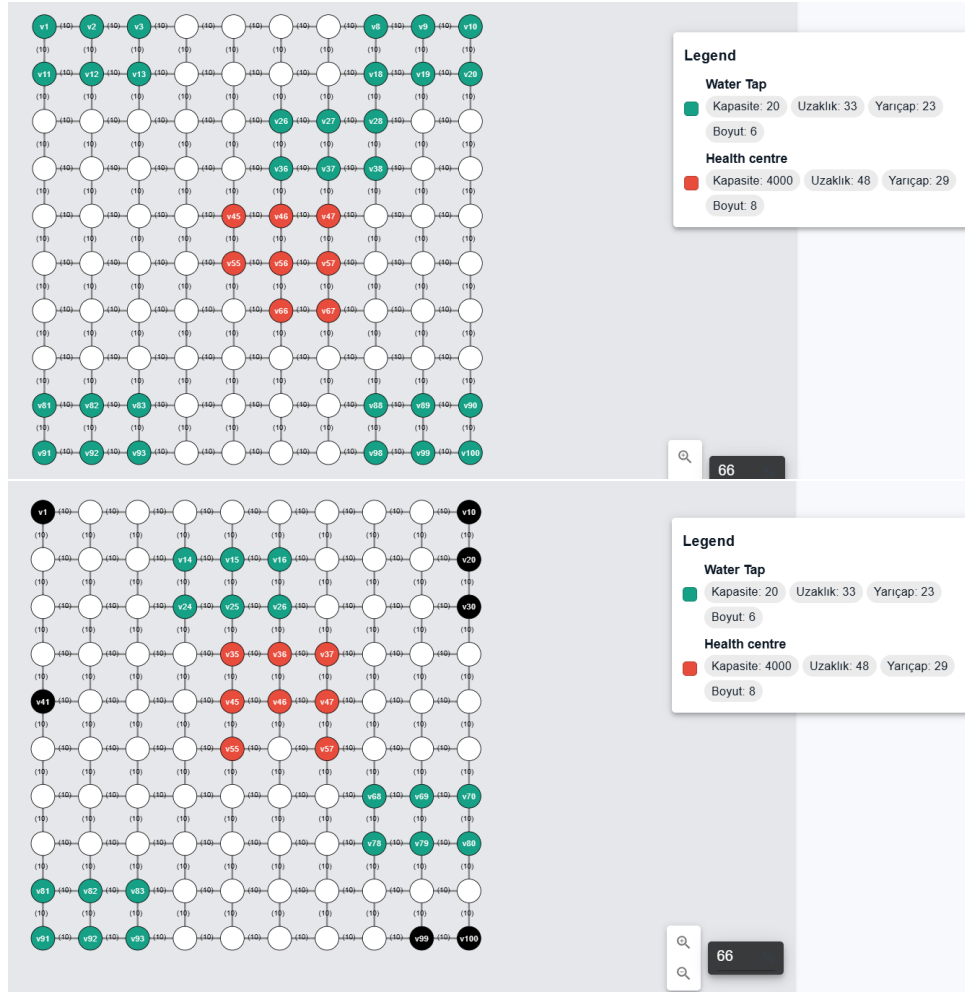


Figure 13: The two none-residential-type with 6 and 8 container size

3. Three none-residential-type



Figure 14: The two none-residential-type with 4, 6 and 8 container size

6.3 Summary:

In summary, the model can obtain an optimum solution for given graphs and parameters. In addition, allowing the model to have empty vertices mostly increases the number of residential-type containers, but never decreases them. Since it removes the force on the model, it does not have to adapt the solution in case of unsuitable container sizes. When the model tries to acquire non-empty-vertex graph, it has to sacrifice some valid residential-type containers to get suitable non-residential-type groups to put them unvalid positions for residential-type containers.